

Sliding Window Order-Recursive Least-Squares Algorithms

Karl Zhao, *Member, IEEE*, Fuyun Ling, *Senior Member, IEEE*, Hanoch Lev-Ari, *Senior Member, IEEE*, and John G. Proakis, *Fellow, IEEE*

Abstract—Order-recursive least-squares (ORLS) algorithms employing a sliding window (SW) are presented. We demonstrate that standard architectures that are well known for growing memory ORLS estimation, e.g., triangular array, lattice, and multichannel lattice, also apply to sliding window ORLS estimation. A specific SW-ORLS algorithm is the combination of two independent attributes: its global architecture and its local cell implementation. Various forms of local cell implementation based on efficient time-recursions of time-varying coefficients are discussed. In particular, we show that time and order updates of any order-recursive sliding window least-squares algorithm can be realized solely in terms of 3×3 hyperbolic Householder transformations (HHT). Finally, we present two HHT-based algorithms: the HHT triangular array algorithm and the HHT lattice algorithm.

I. INTRODUCTION

ADAPTIVE least-squares (LS) algorithms are used to process signals that result from time-varying environments. To facilitate tracking of time-varying parameters, it is desirable to discard old data as new data are collected. This can be achieved by employing windows that weight recent data more heavily than old data. In terms of the memory length of such windows, there are basically two types of algorithms: 1) growing memory LS (GMLS) algorithms and 2) sliding-window LS (SWLS) algorithms. The most widely used are growing memory algorithms whose tracking capability is achieved by employing a weighting scheme that forgets old data exponentially. Although this technique may perform satisfactorily in some cases, its tracking capability is nevertheless limited because old data are never completely discarded. In some applications, it is preferable to use a true finite memory algorithm, i.e., a sliding-window algorithm, in order to avoid undesired effects from data in the remote past. A typical case is where parameters that generate the signal are subject to occasional jump type variations of random amplitudes [1].

Manuscript received May 8, 1992; revised December 6, 1993. This work was supported by NSF Grant MIP-9115526. The associate editor coordinating the review of this paper and approving it for publication was Dr. James Zeidler.

K. Zhao was with the Department of Electrical and Computer Engineering, Northeastern University, Boston, MA 02115 USA. He is currently with DSP Software Engineering, Inc., Bedford, MA 01730 USA.

F. Ling is with Motorola Communications and Electronics Inc., Schaumburg, IL 60196 USA.

H. Lev-Ari and J. G. Proakis are with Department of Electrical and Computer Engineering, Northeastern University, Boston, MA 02115 USA.

IEEE Log Number 9401898.

The classical method of solving LS problems is based on direct solution of the so-called normal equations. In the least-squares estimation problem, this method assumes that the estimator is represented by a transversal structure, which (linearly) combines the input data vector to form an estimate of the desired signal. The normal equations determine the weight vector of the optimal estimator in the sense that the error between the estimate and the true (desired) signal is minimized in the least-squares sense. The order of the estimator is fixed and has to be determined *a priori*. Time-recursive "fast" RLS algorithms that employ a transversal structure are studied in [2]–[4].

In many applications, one is only interested in the LS residual (estimation error) rather than the optimal weight vector. In such a case, the LS residuals of any order can be computed recursively from the lower order residuals. Algorithms that utilize this approach are called order-recursive least-squares (ORLS) algorithms. The advantages of using ORLS algorithms are as follows: 1) They have a modular structure—an array of interconnected identical cells and 2) they can be pipelined, which is due to their feed-forward structure. These two properties make ORLS algorithms well suited for VLSI systolic array implementation. Another desirable property of ORLS algorithms is their numerical robustness, which is due to the following: 1) ORLS algorithms directly operate on the data rather than the covariance matrix, and 2) ORLS algorithms have a feed-forward structure, and there is no global feedback involved.

The conventional RLS algorithm updates the inverse of the sample covariance matrix, whose elements have twice as much dynamic range as the original data and, therefore, a higher condition number. The so-called square-root RLS algorithms [5] propagate the triangular factorization (LU , LDU , UDU') of either the covariance matrix or its inverse instead to avoid the numerically undesirable step of squaring the data. An alternative approach is to carry out a sequence of orthogonal transformations directly on the raw data obtaining a triangular matrix, from which the LS solution can be found simply by back substitution. This approach is motivated by the notion of a QR decomposition (QRD) of a matrix and gives rise to the so-called QR-RLS algorithm [6]. The direct residual extraction QR-RLS algorithm given by McWhirter in [7] is order recursive in nature and is closely related to the other ORLS algorithms [8], [9]. It turns out that QR-RLS algorithms can be also viewed as square-root algorithms

since the triangular matrix propagated in these algorithms is the Cholesky factor of the covariance matrix [10]. The ORLS algorithms, which also belong to the family of square-root algorithms, operate on the data directly and numerically outperform the conventional RLS algorithm.

Transversal-type adaptive algorithms have embedded global feedbacks, which make these algorithms sensitive to the round-off noise [11]. In addition, the global feedback structure makes the stability analysis and round-off analysis of these algorithms rather difficult [4]. On the other hand, there is no global feedback in *order-recursive algorithms*, and therefore, both stability and round-off analysis can be carried out at the level of an individual module. Moreover, once the conditions for the numerical stability of a single module have been established, the stability of the entire algorithm is automatically guaranteed. The modular structure also enables the round-off analysis done on the module level [12].

The structure of ORLS algorithms depends both on the construction of a network of interconnected cells via order recursions and the update of local variables via time recursions, which is affected by the choice of windowing schemes. To be more specific, the LS estimation problem amounts to finding the coefficients $c_i(t)$, $i = 1, \dots, M$ that minimize the cost function

$$J(t) = \sum_{k=0}^t w(k) |e(t-k, t)|^2 \quad (1)$$

where

$$e(k, t) = x(k) - \sum_{i=1}^M y_i(k) c_i(t) \quad (2)$$

where $x(k)$ is the desired signal, and $y_i(k)$ ($i = 1, \dots, M$) are input signals from which an estimate of $x(k)$ is formed. The solution of optimal coefficients in terms of so-called normal equations involves a data matrix whose elements depend on the nature of the input data and the windowing function $w(k)$. In the exponentially weighted growing memory case

$$w(k) = \lambda^k \quad k = 0, \dots, t$$

so that the data matrix becomes

$$\begin{pmatrix} \sqrt{\lambda^t} y_1(0) & \sqrt{\lambda^t} y_2(0) & \dots & \sqrt{\lambda^t} y_M(0) \\ \vdots & \vdots & \vdots & \vdots \\ \sqrt{\lambda} y_1(t-1) & \sqrt{\lambda} y_2(t-1) & \dots & \sqrt{\lambda} y_M(t-1) \\ y_1(t) & y_2(t) & \dots & y_M(t) \end{pmatrix}. \quad (3)$$

On the other hand, in the sliding-window method

$$w(k) = \begin{cases} 1 & 0 \leq k \leq L-1 \\ 0 & \text{otherwise} \end{cases}$$

so that the data matrix is

$$\begin{pmatrix} y_1(t-L+1) & y_2(t-L+1) & \dots & y_M(t-L+1) \\ \vdots & \vdots & \vdots & \vdots \\ y_1(t-1) & y_2(t-1) & \dots & y_M(t-1) \\ y_1(t) & y_2(t) & \dots & y_M(t) \end{pmatrix}. \quad (4)$$

The so-called time-series case (for either one of the windowing methods) corresponds to $y_i(t) = y(t-i+1)$, where $y(t)$ is a single signal. For example, for the time-series case with a growing memory method, the data matrix becomes

$$\begin{pmatrix} \sqrt{\lambda^t} y(0) & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots \\ \sqrt{\lambda} y(t-1) & \sqrt{\lambda} y(t-2) & \dots & \sqrt{\lambda} y(t-M) \\ y(t) & y(t-1) & \dots & y(t-M+1) \end{pmatrix}. \quad (5)$$

Notice the data prior to the initial time is set to zero, i.e., $y(k) = 0$ for $k < 0$, which is often known as the prewindowed method.

In spite of the numerous differences in the details of the derivation of the ORLS algorithms for these various windowing schemes, the resulting architectures are essentially the same. This is because that all the windowing schemes discussed above obey the same order-recursive rule. The construction of a global architecture via pure order-recursive arguments is by now well understood [13]–[21]. All ORLS work mentioned above concentrated on the growing window case. The necessity of downdating in the sliding window case complicates the derivation of efficient time recursions. In this paper, we use an embedding approach to show that the sliding window method is closely related to the growing window techniques. This makes it possible to obtain sliding-window recursions by a simple modification of those used in the well-known growing window case.

Our result firmly establishes the two-attribute classification of ORLS algorithms proposed in [21] while extending it to the sliding window case. A complete classification of ORLS algorithms consists of two independent aspects, where each affects a different level of detail:

- At the system (global) level, the algorithm is specified as a network of interconnected modules, each one representing a single first-order LS estimation step. The topology of this network is determined by the nature of the least-square estimation, i.e., whether it is based on a single or multiple channel and whether it uses only present or both present and past samples. The global architecture is constructed via pure order-recursive arguments.
- At the module (local) level, the time-recursive argument is used to update the time-varying coefficients. The time recursions are determined by the choice of windowing schemes. Even with the same windowing scheme, there exist a wide variety of cell implementation.

These two aspects are completely independent in the sense that every choice of architecture can be combined with every choice of a window scheme and every particular cell implementation.

The paper is organized as follows: In Section II, we formulate the adaptive sliding window LS estimation problem. In Section III, we describe some global architectures based on the basic one-step order-update recursion. Various forms of local cell implementation based on efficient time-recursions of time-varying coefficients are discussed in Section IV. In particular, we show that time and order updates of any order-recursive

sliding-window least-squares algorithm can be realized solely in terms of 3×3 hyperbolic Householder transformations (HHT). In Section V, we outline the steps to obtain a particular SW-ORLS algorithm and present two HHT-based algorithms: the HHT triangular array algorithm and the HHT lattice algorithm.

II. BASICS OF SWLS ESTIMATION

In the sliding window LS (SWLS) estimation problem, our objective is to choose the coefficient vector $\mathbf{c}_x(t) = [c_1(t) \ c_2(t) \ \cdots \ c_M(t)]^t$ such that it minimizes the sum of the magnitude squared errors

$$J(t, L) = \sum_{k=0}^{L-1} |e(t-k, t)|^2 \quad (6)$$

where L is the length of the window. From this formulation, it is clear that only errors falling into a window span from time $t-L+1$ to time t contribute to the cost function $J(t, L)$.

A geometric interpretation of (6) is obtained by introducing a (Euclidean) space of data vectors and, in particular, constructing the data matrix $\mathbf{A}(t)$, the desired signal vector $\mathbf{x}(t)$, and the error vector $\mathbf{e}_x(t)$ as follows:

$$\mathbf{A}(t) = \begin{pmatrix} y_1(t-L+1) & y_2(t-L+1) & \cdots & y_M(t-L+1) \\ \vdots & \vdots & \ddots & \vdots \\ y_1(t) & y_2(t) & \cdots & y_M(t) \end{pmatrix} \\ = [\mathbf{a}_1(t), \mathbf{a}_2(t), \dots, \mathbf{a}_M(t)] \quad (7)$$

$$\mathbf{x}(t) = [x(t-L+1), \dots, x(t)]^t \quad (8)$$

and

$$\mathbf{e}_x(t) = [e_x(t-L+1, t), \dots, e_x(t, t)]^t. \quad (9)$$

Consequently, $\mathbf{e}_x(t)$ can be expressed as

$$\mathbf{e}_x(t) = \mathbf{x}(t) - \mathbf{A}(t)\mathbf{c}_x(t) \quad (10)$$

and the minimization of $J(t, L)$ is equivalent to minimizing the Euclidean norm of $\mathbf{e}_x(t)$, i.e.

$$J(t, L) = \|\mathbf{e}_x(t)\|^2 \quad (11)$$

where $\|\cdot\|$ denotes the Euclidean norm of an L -dimensional vector. Using (10), it is easy to show that the minimization of $J(t, L)$ as expressed in (11) leads to the normal equations

$$\mathbf{c}_x(t) = [\mathbf{A}^H(t)\mathbf{A}(t)]^{-1}\mathbf{A}^H(t)\mathbf{x}(t) \quad (12)$$

where $\mathbf{c}_x(t)$ is the optimal solution to (6).

In adaptive signal processing applications, we wish to obtain the SWLS solution in a time-recursive manner. Specifically, we wish to calculate quantities at time t from the corresponding quantities at time $t-1$. In time-recursive GMLS estimation, we only need to update quantities with respect to the new data at time t . On the other hand, in time-recursive SWLS estimation, we also need to downdate with respect to the old data from time $t-L$, which are no longer part of the data matrix. The structure of all time-recursive solutions to the SWLS estimation problem is determined by the need

to carry out these two elementary operations (updating and downdating) for every time instant and for every quantity whose time evolution we wish to propagate. Some authors prefer to carry out the update and the downdate separately [1]. In this paper, we combine both elementary operations into a single step that uses the notion of hyperbolic transformations.

An efficient implementation of the update step in growing memory LS estimation is made possible by the observation that the new data involved in the update are the last elements of the data vectors at time t . The same observation holds true for the update step in sliding-window LS estimation but not so for the downdate step: The latter involves data that are (the first) elements of the data vectors at time $t-1$ but not of those at time t . This significantly complicates the derivation of the corresponding time-recursion expressions [1]. In what follows, we introduce an augmented vector space interpretation for the sliding window LS solution (12), which avoids this difficulty and makes the SWLS recursions completely analogous to the better understood GMLS recursions.

We construct an augmented vector space of $L+2$ dimensions by adding data at $t-L$ on the top and bottom of the corresponding L -dimensional vector previously defined. We denote the augmented quantities by an upper bar, e.g., the augmented data matrix becomes

$$\bar{\mathbf{A}}(t) = \begin{pmatrix} y_1(t-L) & y_2(t-L) & \cdots & y_M(t-L) \\ & & & \mathbf{A}(t) \\ y_1(t-L) & y_2(t-L) & \cdots & y_M(t-L) \end{pmatrix} \quad (13)$$

$$= [\bar{\mathbf{a}}_1(t), \bar{\mathbf{a}}_2(t), \dots, \bar{\mathbf{a}}_M(t)]. \quad (14)$$

Notice that $\bar{\mathbf{A}}(t)$ can also be viewed as the data matrix obtained by adding two rows at the bottom of $\mathbf{A}(t-1)$, viz.

$$\bar{\mathbf{A}}(t) = \begin{pmatrix} \mathbf{A}(t-1) \\ \mathbf{Y}(t) \end{pmatrix} \\ = \begin{pmatrix} \mathbf{a}_1(t-1) & \mathbf{a}_2(t-1) & \cdots & \mathbf{a}_M(t-1) \\ \bar{\mathbf{a}}_1(t) & \bar{\mathbf{a}}_2(t) & \cdots & \bar{\mathbf{a}}_M(t) \end{pmatrix} \quad (15)$$

which shows explicitly the dependence on the two data at time t and at $t-L$ used in the updating and downdating steps, namely

$$\mathbf{Y}(t) = [\bar{\mathbf{a}}_1(t) \ \bar{\mathbf{a}}_2(t) \ \cdots \ \bar{\mathbf{a}}_M(t)] \quad (16)$$

$$= \begin{pmatrix} y_1(t) & y_2(t) & \cdots & y_M(t) \\ y_1(t-L) & y_2(t-L) & \cdots & y_M(t-L) \end{pmatrix}. \quad (17)$$

Similarly, the augmented $\bar{\mathbf{x}}(t)$ and $\bar{\mathbf{e}}_x(t)$ are defined as

$$\bar{\mathbf{x}}(t) = \begin{pmatrix} x(t-L) \\ \mathbf{x}(t) \\ x(t-L) \end{pmatrix} = \begin{pmatrix} \mathbf{x}(t-1) \\ \mathbf{x}(t) \end{pmatrix} \quad (18)$$

and

$$\bar{\mathbf{e}}_x(t) = \begin{pmatrix} e(t-L, t) \\ \mathbf{e}_x(t) \\ e(t-L, t) \end{pmatrix} = \begin{pmatrix} \mathbf{e}_x(t-1) \\ \mathbf{e}_x(t) \end{pmatrix} \quad (19)$$

respectively. In analogy with (10), we can express $\bar{\mathbf{e}}_x(t)$ as

$$\bar{\mathbf{e}}_x(t) = \bar{\mathbf{x}}(t) - \bar{\mathbf{A}}(t)\mathbf{c}_x(t) \quad (20)$$

and consequently,

$$\underline{\mathbf{e}}_x(t) = \underline{\mathbf{x}}(t) - \mathbf{Y}(t)\mathbf{c}_x(t). \quad (21)$$

The underlined symbols $\underline{\mathbf{x}}(t)$ and $\underline{\mathbf{e}}_x(t)$, which are defined in (21), denote quantities involved in updating and downdating. In particular, $\underline{\mathbf{e}}_x(t)$ is called the *a posteriori* error of SWLS estimation.

In order to relate our augmented vectors to the SWLS formulation (11), we replace the conventional Euclidean norm $\|\cdot\|$ used with the L -dimensional vectors by a signed (or weighted) norm $\|\cdot\|_{\mathbf{W}}$ to be used with the $(L+2)$ -dimensional vectors. In particular

$$J(t, L) = \|\underline{\mathbf{e}}_x(t)\|_{\mathbf{W}}^2 = \underline{\mathbf{e}}_x^H(t) \mathbf{W} \underline{\mathbf{e}}_x(t) \quad (22)$$

where $\mathbf{W} = \text{diag}(1, \dots, 1, -1)$. In general, $\|\cdot\|_{\mathbf{W}}$ is signed (i.e., not necessarily positive) norm in the $(L+2)$ -dimensional vector space since there exist vectors whose norms are negative. However, $\|\cdot\|_{\mathbf{W}}$ defines a positive norm for the subspace spanned by the augmented vectors in $(L+2)$ -dimensional vector space because it follows from (11) and (22) that the Euclidean norm of a vector in the original L -dimensional space is the same as the weighted norm of its corresponding augmented vector in the $(L+2)$ -dimensional space. Thus, the normal equations (12) can be expressed in terms of augmented vectors and weighted norms, i.e.

$$\begin{aligned} \mathbf{c}_x(t) &= [\bar{\mathbf{A}}^H(t) \mathbf{W} \bar{\mathbf{A}}(t)]^{-1} \bar{\mathbf{A}}^H(t) \mathbf{W} \bar{\mathbf{x}}(t) \\ &= \mathbf{R}^{-1}(t) \mathbf{q}_x(t). \end{aligned} \quad (23)$$

We can now use a similar approach employed in conventional growing memory ORLS algorithms to derive time-recursive SWLS algorithms. This point will become more clear later when we discuss time recursions.

III. GLOBAL ARCHITECTURES OF SW-ORLS ALGORITHMS

The global architecture is mainly determined by the way in which the order recursions are performed. We first establish a one-step order-update recursion of SWLS estimation problem, which is used as a basic module to construct a network of interconnected modular cells.

In SW-ORLS estimation, $\bar{\mathbf{e}}_x(t)$ is computed without obtaining $\mathbf{c}_x(t)$ explicitly. From (20) and (23), we have

$$\bar{\mathbf{e}}_x(t) = \bar{\mathbf{x}}(t) - \bar{\mathbf{A}}(t) \mathbf{R}^{-1}(t) \mathbf{q}_x(t). \quad (24)$$

An order update of $\bar{\mathbf{e}}_x(t)$ amounts to replacing the data matrix $\bar{\mathbf{A}}(t)$ in (24) by the higher order data matrix $\tilde{\mathbf{A}}(t) = [\bar{\mathbf{A}}(t) : \bar{\mathbf{z}}(t)]$, where $\bar{\mathbf{z}}(t)$ denotes an additional data channel, i.e., an additional column of $\bar{\mathbf{A}}(t)$ to be considered along with the other channels (or columns) $\bar{\mathbf{a}}_1(t)$, $\bar{\mathbf{a}}_2(t)$, $\dots$, $\bar{\mathbf{a}}_M(t)$. The corresponding error-updated estimation error vector, which we denote by $\bar{\mathbf{e}}_x^z(t)$, is related to $\bar{\mathbf{e}}_x(t)$ by

$$\bar{\mathbf{e}}_x^z(t) = \bar{\mathbf{e}}_x(t) - k_{xz}(t) \bar{\mathbf{e}}_z(t) = \bar{\mathbf{e}}_x(t) - \frac{r_{xz}(t)}{r_{zz}(t)} \bar{\mathbf{e}}_z(t) \quad (25)$$

where $\bar{\mathbf{e}}_z(t)$ denotes the error of estimating $\bar{\mathbf{z}}(t)$ in terms of the data in $\bar{\mathbf{A}}(t)$; $r_{xz}(t)$ is the cross-correlation between error vectors $\bar{\mathbf{e}}_x(t)$ and $\bar{\mathbf{e}}_z(t)$, which are defined as $r_{xz}(t) =$

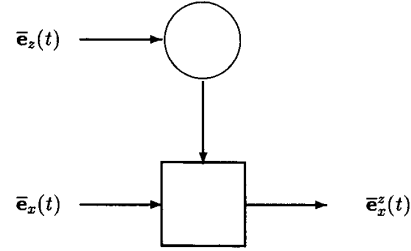


Fig. 1. Vector form one-step order-update module.

$\bar{\mathbf{e}}_z^H(t) \mathbf{W} \bar{\mathbf{e}}_x(t)$, and $r_{zz}(t)$ is the autocorrelation of the error vector $\bar{\mathbf{e}}_z(t)$, which is defined as $r_{zz}(t) = \bar{\mathbf{e}}_z^H(t) \mathbf{W} \bar{\mathbf{e}}_z(t)$.

Both (24) and (25) have a similar geometrical interpretation: The former describes $\bar{\mathbf{e}}_x(t)$ as the residual obtained by projecting $\bar{\mathbf{x}}(t)$ on the space spanned by the column of $\bar{\mathbf{A}}(t)$, i.e., $\bar{\mathbf{a}}_1(t)$, $\bar{\mathbf{a}}_2(t)$, $\dots$, $\bar{\mathbf{a}}_M(t)$, and the latter describes $\bar{\mathbf{e}}_x^z(t)$ as the residual obtained by projecting $\bar{\mathbf{e}}_x(t)$ on the single vector $\bar{\mathbf{e}}_z(t)$. The main difference is that (24) involves matrix operations, whereas (25) can be viewed as first-order SWLS estimation, which involves only scalar operations. The order recursion can be expressed by the notion of projection

$$\bar{\mathbf{e}}_x^z(t) = \bar{\mathbf{e}}_x(t) |_{\bar{\mathbf{e}}_z(t) = \bar{\mathbf{x}}(t)} |_{\{\bar{\mathbf{A}}(t), \bar{\mathbf{z}}(t)\}}. \quad (26)$$

The basic (vector form) order-recursive module is illustrated in Fig. 1. The cell represented by a circle is called the reference cell since it provides the reference signal $\bar{\mathbf{e}}_x(t)$ for the actual projection. The cell represented by a square carries out the actual projection called the projection cell.

For the purpose of deriving algorithms that have modular structures, it is desirable to construct SW-ORLS estimations of any given order, such as $\bar{\mathbf{e}}_x(t)$, by repeated one-step updates like (25), which performs a scalar SWLS estimation. Thus, (25) is the fundamental relation for deriving SW-ORLS algorithms. We should also point out the basic condition of (25) is that $\bar{\mathbf{e}}_x(t)$ and $\bar{\mathbf{e}}_z(t)$ must be SWLS errors based on the subspace spanned by the same set of data.

The global architecture of SW-ORLS algorithms is determined by the nature of the least-square estimation, i.e., by the structure of the input vector. In array processing applications, the input data vector is derived from an array of multiple sensors, the subspace decomposition leads to a triangular array structure, and the size of the array is proportional to M^2 (or $O(M^2)$). In adaptive filtering applications, the input vector is derived from a tap-delayed line. By using the shift-invariant property of the time-series input, efficient order-recursive schemes can be used to a lattice-ladder structure whose size is reduced to $O(M)$. In the multichannel filtering applications, multichannel lattice structures can be obtained by taking the advantage of the block shift-invariant property of the input vector.

A. Triangular Array Architecture

We consider the SW-ORLS estimation problem described in Section II. However, we wish to obtain the M th order error by performing multiple scalar estimations. We define a growing

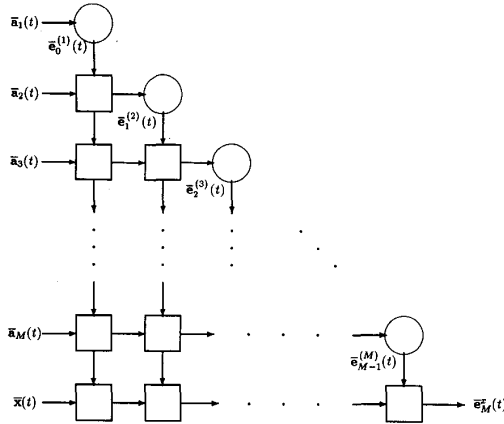


Fig. 2 Triangular array architecture.

order subspace $\mathcal{U}_m(t)$ spanned by the augmented data vectors $\{\bar{\mathbf{a}}_i(t), 1 \leq i \leq m\}$, which are expressed as

$$\mathcal{U}_m(t) = \{\bar{\mathbf{a}}_1(t), \bar{\mathbf{a}}_2(t), \dots, \bar{\mathbf{a}}_m(t)\} \quad (27)$$

where $1 \leq m \leq M$, and $\mathcal{U}_0 = \mathbf{0}$. The m th-order estimation error is defined as $\bar{\mathbf{e}}_m^x(t) = \bar{\mathbf{x}}(t) |_{\mathcal{U}_m(t)}$. We also define $\bar{\mathbf{e}}_m^{(i)}(t) = \bar{\mathbf{a}}_i(t) |_{\mathcal{U}_m(t)}$ for $m+1 \leq i \leq M$. Notice that all these errors (vector) are based on the same subspace. To obtain the $(m+1)$ th-order error, we need simply project $\bar{\mathbf{e}}_m^x(t)$ onto $\bar{\mathbf{e}}_m^{(m+1)}(t)$

$$\bar{\mathbf{e}}_{m+1}^x(t) = \bar{\mathbf{e}}_m^x(t) |_{\bar{\mathbf{e}}_m^{(m+1)}(t)} = \bar{\mathbf{x}}(t) |_{\{\mathcal{U}_m(t), \bar{\mathbf{a}}_{m+1}(t)\}}. \quad (28)$$

We also need to obtain $\bar{\mathbf{e}}_{m+1}^{(i)}(t)$ for the next order recursion, which can be achieved by projecting $\bar{\mathbf{e}}_m^{(i)}(t)$ onto $\bar{\mathbf{e}}_m^{(m+1)}(t)$, $m+2 \leq i \leq M$, i.e.

$$\bar{\mathbf{e}}_{m+1}^{(i)}(t) = \bar{\mathbf{e}}_m^{(i)}(t) |_{\bar{\mathbf{e}}_m^{(m+1)}(t)} = \bar{\mathbf{a}}_i(t) |_{\{\mathcal{U}_m(t), \bar{\mathbf{a}}_{m+1}(t)\}}. \quad (29)$$

Clearly, at the m th stage, it requires $M-m$ numbers of scalar SWLS estimation, but all of them use the common reference signal $\bar{\mathbf{e}}_m^{(m+1)}(t)$. Therefore, in a time-recursive implementation, we can compute the common parameters in the reference cell and propagate these parameters through all projection cells using this same set of parameters. By repeating this procedure for $0 \leq m \leq M-1$, we can obtain $\bar{\mathbf{e}}_M^x(t)$. It is straightforward to map this procedure onto a triangular array shown in Fig. 2.

This array has a clear geometric interpretation, which resembles the modified Gram-Schmidt orthogonalization procedure. The signals that are inputs of boundary circular cells, i.e., $\bar{\mathbf{e}}_m^{(m+1)}(t)$, $0 \leq m \leq M-1$, form an orthogonal basis of the space $\mathcal{U}_M(t)$. The square cells in the right-most column of the array project the desired signal successively onto this basis; other projection cells in the triangular array construct the basis in the procedure described above.

B. Lattice Linear Array Architecture

In this section, we assume that the data vector is a segment of a time series such that

$$\mathbf{y}(t) = [y(t), y(t-1), \dots, y(t-M+1)]^t. \quad (30)$$

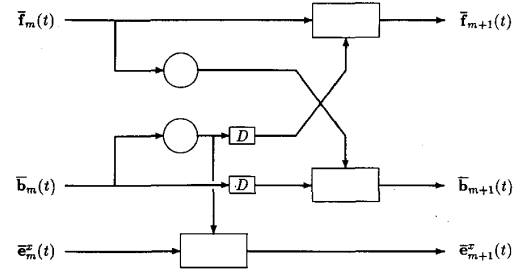


Fig. 3 One stage of lattice-ladder architecture.

At time $t+1$, the input vector $\mathbf{y}(t+1)$ is a shifted version of $\mathbf{y}(t)$ with the next available data element being appended to the shift. Hence, we say that the data vector $\mathbf{y}(t)$ in (30) has a “time-shifting” property. We denote the augmented data vector as

$$\bar{\mathbf{a}}(t) = [y(t-L), y(t-L+1), \dots, y(t), y(t-L)]^t. \quad (31)$$

In such a case, the m th-order data subspace becomes

$$\mathcal{U}_m(t) = \{\bar{\mathbf{a}}(t), \bar{\mathbf{a}}(t-1), \dots, \bar{\mathbf{a}}(t-m+1)\}. \quad (32)$$

The orthogonal basis of $\mathcal{U}_M(t)$ in the case of a time-series signal input consists of the SWLS errors of $\bar{\mathbf{a}}(t-m)$ based on $\mathcal{U}_m(t)$ (for all $0 \leq m \leq M-1$), which are denoted as $\bar{\mathbf{b}}_m(t) = \bar{\mathbf{a}}(t-m) |_{\mathcal{U}_m(t)}$, which we call the m th-order backward prediction error. Now, with the special property of shift invariance, we may construct this basis by avoiding the use of a triangular array. This is achieved by introducing forward prediction along with backward prediction. The m th forward prediction error is defined as the SWLS estimation error of $\bar{\mathbf{a}}(t)$ based on $\mathcal{U}_m(t-1)$ and denoted as $\bar{\mathbf{f}}_m(t) = \bar{\mathbf{a}}(t) |_{\mathcal{U}_m(t-1)}$.

It is clear from this definition that $\bar{\mathbf{f}}_m(t)$ and $\bar{\mathbf{b}}_m(t-1)$ are SWLS errors based on the same subspace $\mathcal{U}_m(t-1)$. Now, let us project $\bar{\mathbf{f}}_m(t)$ onto $\bar{\mathbf{b}}_m(t-1)$

$$\bar{\mathbf{f}}_m(t) |_{\bar{\mathbf{b}}_m(t-1)} = \bar{\mathbf{a}}(t) |_{\{\mathcal{U}_m(t-1), \bar{\mathbf{a}}(t-m-1)\}} = \bar{\mathbf{f}}_{m+1}(t) \quad (33)$$

and thus, we obtain the $(m+1)$ th forward prediction error. Similarly, we can update backward prediction error by projecting $\bar{\mathbf{b}}_m(t-1)$ onto $\bar{\mathbf{f}}_m(t)$, i.e.

$$\bar{\mathbf{b}}_m(t-1) |_{\bar{\mathbf{f}}_m(t)} = \bar{\mathbf{a}}(t-m-1) |_{\{\mathcal{U}_m(t-1), \bar{\mathbf{a}}(t)\}} = \bar{\mathbf{b}}_{m+1}(t).$$

The $\bar{\mathbf{e}}_m^x(t)$ and $\bar{\mathbf{b}}_m(t)$ are SWLS errors based on the same subspace $\mathcal{U}_m(t)$. By projecting $\bar{\mathbf{e}}_m^x(t)$ onto $\bar{\mathbf{b}}_m(t)$, we obtain higher order estimation error $\bar{\mathbf{e}}_{m+1}^x(t)$:

$$\bar{\mathbf{e}}_m^x(t) |_{\bar{\mathbf{b}}_m(t)} = \bar{\mathbf{x}}(t) |_{\{\mathcal{U}_m(t), \bar{\mathbf{a}}(t-m)\}} = \bar{\mathbf{e}}_{m+1}^x(t). \quad (34)$$

This step is usually called joint estimation.

Therefore, a lattice-type linear array consists of a cascade of identical lattice stages. The basic function of each lattice stage is to perform three order-update recursions. Thus, the outputs of each lattice stage are a set of three SWLS errors generated from another set of SWLS errors of lower order, which is illustrated in Fig. 3.

C. Multichannel Lattice Architectures

Now, we consider the input vector of the form

$$\mathbf{y}(t) = [y_1(t), y_2(t), \dots, y_p(t), y_1(t-1), \dots, \dots, y_1(t-N+1), \dots, y_p(t-N+1)]^t \quad (35)$$

where $M = pN$. It has the block shifting property with the block defined as

$$\bar{\mathbf{B}}(t) = [\bar{\mathbf{a}}_1(t), \bar{\mathbf{a}}_2(t), \dots, \bar{\mathbf{a}}_p(t)]. \quad (36)$$

Now, we define the following subspace:

$$\mathcal{U}_m(t) = \{\bar{\mathbf{B}}(t), \bar{\mathbf{B}}(t-1), \dots, \bar{\mathbf{B}}(t-m+1)\} \quad 0 \leq m \leq N-1. \quad (37)$$

By using $\bar{\mathbf{B}}(t)$ in place of $\bar{\mathbf{a}}(t)$ and following the same development as in the SWLS lattice algorithm given in the previous section, we obtain a p -channel SWLS lattice algorithm that has $N-1$ stages. However, since there are p channels, the order recursions of each lattice stage takes the block form. Using the same idea as in previous sections, we can construct the block form order updates by repeatedly using scalar SWLS estimations. Two approaches have been used to develop modular GMLS multichannel lattice stages: one update p -channel forward and backward error separately [17] and the other jointly [18]. Since the architecture is independent on the criteria used, these two approaches can both be employed to obtain the SWLS multichannel lattice stages.

In the first approach, we utilize the following notion of forward and backward error definitions. The forward error $\bar{\mathbf{f}}_m^{(i)}(t)$ is the error of $\bar{\mathbf{a}}_i(t)$ based on $\mathcal{U}_m(t-1)$, and the backward error $\bar{\mathbf{b}}_m^{(i)}(t)$ is the SWLS error of $\bar{\mathbf{a}}_i(t-m)$ based on $\mathcal{U}_m(t)$, where the index i denotes the channel $1 \leq i \leq p$. Clearly, the $\bar{\mathbf{f}}_m^{(i)}(t)$ and $\bar{\mathbf{b}}_m^{(i)}(t-1)$ are based on the same subspace. The forward errors can be updated by projecting $\bar{\mathbf{f}}_m^{(i)}(t)$ onto the space spanned by $\bar{\mathbf{b}}_m^{(i)}(t-1)$ $1 \leq i \leq p$. This is equivalent to viewing $\bar{\mathbf{f}}_m^{(i)}(t)$ as the desired signal and $\bar{\mathbf{b}}_m^{(i)}(t-1)$ as the data signal in the triangular array. There are p such desired signals $\bar{\mathbf{f}}_m^{(i)}(t)$, $1 \leq i \leq p$. Consequently, the forward update is a triangular array followed by a square, as illustrated in Fig. 4. A similar structure also applies to the backward update.

The second approach defines the forward and backward errors in a different way. Now, let us introduce another symbol to denote subspace with growing order in the channel

$$\mathcal{V}_i(t) = \{\bar{\mathbf{a}}_1(t), \bar{\mathbf{a}}_2(t), \dots, \bar{\mathbf{a}}_i(t)\} \quad 1 \leq i \leq p. \quad (38)$$

We define the following prediction errors:

$$\bar{\mathbf{f}}_m^{(i,j)}(t) = \bar{\mathbf{a}}_i(t) | \{\mathcal{U}_m(t-1), \mathcal{V}_{i-1}(t), \mathcal{V}_j(t-m-1)\} \quad (39)$$

$$\bar{\mathbf{b}}_m^{(i,j)}(t) = \bar{\mathbf{a}}_i(t-m-1) | \{\mathcal{U}_m(t-1), \mathcal{V}_i(t), \mathcal{V}_{j-1}(t-m-1)\}. \quad (40)$$

Now, $\bar{\mathbf{f}}_m^{(i,j-1)}(t)$ and $\bar{\mathbf{b}}_m^{(i-1,j)}(t)$ are errors based on the same subspace, and thus, they can be updated with each other.

$$\bar{\mathbf{f}}_m^{(i,j)}(t) = \bar{\mathbf{f}}_m^{(i,j-1)}(t) |_{\bar{\mathbf{b}}_m^{(i-1,j)}(t)} \quad (41)$$

$$\bar{\mathbf{b}}_m^{(i,j)}(t) = \bar{\mathbf{b}}_m^{(i-1,j)}(t) |_{\bar{\mathbf{f}}_m^{(i,j-1)}(t)}. \quad (42)$$

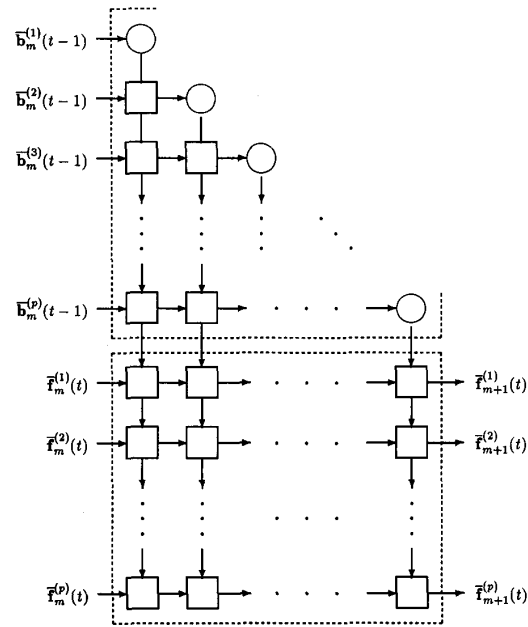


Fig. 4. Multichannel lattice stage based on MGS.

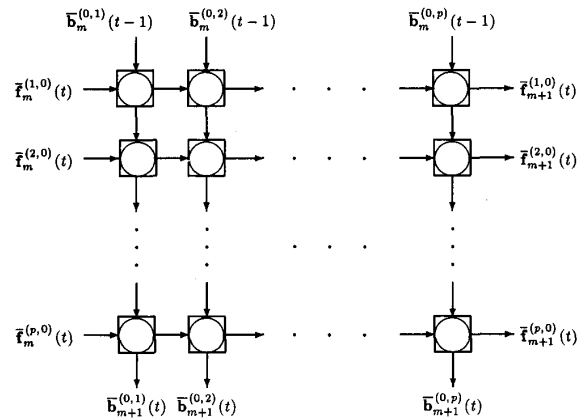


Fig. 5. Multichannel lattice stage based on biorthogonalization.

This forms a square array, where $\bar{\mathbf{f}}_m^{(i,j)}(t)$ and $\bar{\mathbf{b}}_m^{(i,j)}(t)$ denote the outputs of the cell on the i th row and j th column. Each cell is a single-channel lattice stage (without the joint-estimation part). The left and top boundary cells of the m th-order multichannel stage takes the input from the right and bottom boundary cells of the previous stage

$$\begin{aligned} \bar{\mathbf{f}}_m^{(i,0)}(t) &= \bar{\mathbf{f}}_{m+1}^{(i,p)}(t) \\ \bar{\mathbf{b}}_m^{(0,j)}(t) &= \bar{\mathbf{b}}_{m+1}^{(p,j)}(t-1) \quad 1 \leq i, j \leq p. \end{aligned} \quad (43)$$

The resulting architecture is a square array of interconnected single-channel lattice stages, which is the same as the GMLS multichannel lattice arrays shown in [18] and [20]. One stage of the multichannel array is shown in Fig. 5.

IV. LOCAL CELL IMPLEMENTATION OF SW-ORLS ALGORITHMS

The time-varying coefficients in the order-recursive relations can be computed time recursively, namely, in order to obtain the coefficient $r_{xz}(t)$, it is not necessary to recompute the entire error vectors $\bar{\mathbf{e}}_x(t)$ and $\bar{\mathbf{e}}_z(t)$. The new value could be computed from its previous value and their last two elements associated with the updating and downdating due to the nature of the sliding window scheme.

A. A Posteriori Error Form Cells

By using the embedding technique described in Section II, we can derive the time-recursions following the same approach used in the growing memory case [22]. The time recursion for $r_{xz}(t)$ in terms of the *a posteriori* errors is given by

$$r_{xz}(t) = r_{xz}(t-1) + \mathbf{e}_z^H(t) \Phi^{-1}(t) \mathbf{e}_x(t). \quad (44)$$

The time recursion for the autocorrelation $r_{zz}(t)$ can be obtained in a similar manner, which is

$$r_{zz}(t) = r_{zz}(t-1) + \mathbf{e}_z^H(t) \Phi^{-1}(t) \mathbf{e}_z(t) \quad (45)$$

where $\Phi(t)$ is a Hermitian 2×2 matrix defined as

$$\Phi(t) = \phi_2 - \mathbf{Y}(t) \mathbf{R}^{-1}(t) \mathbf{Y}^H(t) \quad (46)$$

where $\phi_2 = \text{diag}(1, -1)$. Since the matrix $\Phi(t)$ is the SWLS equivalent of the so-called likelihood variable, which is a scalar in the GMLS case, we shall call it the likelihood matrix.

To accomplish the time-recursions of $r_{xz}(t)$ and $r_{zz}(t)$, it is sufficient to propagate the last two elements in the error vector, i.e., the *a posteriori* error. Since (25) is an order recursion for vectors, it holds, in particular, for the last two elements of each vector, i.e.

$$\mathbf{e}_x^z(t) = \mathbf{e}_x(t) - k_{xz}(t) \mathbf{e}_z(t) \quad (47)$$

where $\mathbf{e}_x(t)$ denotes the last two elements of $\bar{\mathbf{e}}_x(t)$ and similarly for $\mathbf{e}_z(t)$ and $\mathbf{e}_x^z(t)$.

To cascade the basic time-recursive SWLS scalar modules, we also need to compute the higher order likelihood matrix $\tilde{\Phi}(t)$ associated with the higher order data matrix $\tilde{\mathbf{A}}(t) = [\tilde{\mathbf{A}}(t) \tilde{\mathbf{z}}(t)]$. This can be achieved by updating $\Phi(t)$ with $\mathbf{e}_z(t)$ or $\mathbf{e}_x^z(t)$. The corresponding order recursion for the likelihood matrix is given by

$$\tilde{\Phi}(t) = \Phi(t) - \frac{1}{r_{zz}(t)} \mathbf{e}_z(t) \mathbf{e}_z^H(t). \quad (48)$$

Thus, (44), (45), (47), and (48) form the basic equation set for the *a posteriori* form SW-ORLS estimation. We can implement the basic *a posteriori* SW-ORLS equation set using two types of processing cells as depicted in Fig. 6. The reference cell implements (45) and (48), and the projection cell implements (44) and (47).

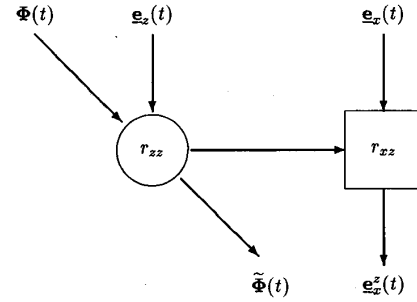


Fig. 6. Basic processing cells of SW-ORLS estimation.

B. A Priori Error Form Cells

The correlations can also be updated by propagating the so-called *a priori* residuals $\hat{\mathbf{e}}_x(t)$, $\hat{\mathbf{e}}_z(t)$, and $\hat{\mathbf{e}}_x^z(t)$, which are the residuals at time t calculated using the optimal coefficients at time $t-1$, e.g.

$$\hat{\mathbf{e}}_x(t) = \mathbf{x}(t) - \mathbf{Y}(t) \mathbf{c}_x(t-1) \quad (49)$$

instead of the *a posteriori* residuals $\bar{\mathbf{e}}_x(t)$, $\bar{\mathbf{e}}_z(t)$, and $\bar{\mathbf{e}}_x^z(t)$. The *a priori* equivalent of the order recursion (47) is

$$\hat{\mathbf{e}}_x^z(t) = \hat{\mathbf{e}}_x(t) - k_{xz}(t-1) \hat{\mathbf{e}}_z(t). \quad (50)$$

The *a posteriori* and the *a priori* errors are related by

$$\mathbf{e}_x(t) = \Phi(t) \phi_2 \hat{\mathbf{e}}_x(t). \quad (51)$$

The time recursion for $r_{xz}(t)$ in terms of the *a priori* errors is given by

$$r_{xz}(t) = r_{xz}(t-1) + \hat{\mathbf{e}}_z^H(t) \phi_2 \Phi(t) \phi_2 \hat{\mathbf{e}}_x(t) \quad (52)$$

and similarly

$$r_{zz}(t) = r_{zz}(t-1) + \hat{\mathbf{e}}_z^H(t) \phi_2 \Phi(t) \phi_2 \hat{\mathbf{e}}_z(t). \quad (53)$$

The update of maximum likelihood matrix in terms of *a priori* error can be expressed as

$$\tilde{\Phi}(t) = \Phi(t) - \frac{1}{r_{zz}(t)} \Phi(t) \phi_2 \hat{\mathbf{e}}_z(t) \hat{\mathbf{e}}_z^H(t) \phi_2 \Phi(t). \quad (54)$$

In [23], it has been found that for growing memory ORLS algorithms, correlation quotient forms are sensitive to round-off error. The so-called error-feedback form for directly time updating the LS coefficient was introduced there to improve the numerical accuracy in the steady state. Using (50), (52), and (53), one can show that the *a priori* error-feedback form for directly updating the SWLS coefficient $k_{xz}(t)$ from

$k_{xz}(t-1)$ without using $r_{xz}(t)$ is given by

$$k_{xz}(t) = k_{xz}(t-1) - \frac{1}{r_{zz}(t)} \hat{\mathbf{e}}_z(t) \phi_2 \Phi(t) \phi_2 \hat{\mathbf{e}}_x^z(t). \quad (55)$$

Equations (50), (52) or (55), (53), and (54) form the basic equation set for the *a priori* form SW-ORLS estimation. The reference cell implements (53) and (54), and the projection cell implements (50) and (52) or (55).

C. Angle-Normalized Error Form

In both the *a posteriori* and the *a priori* error forms, one needs to propagate the likelihood matrix variable. In the growing memory case, the so-called angle-normalized error forms are used to avoid the propagation of the likelihood variable. In this case, both time and order updates can be achieved by a single 2×2 Givens rotation. Similarly, we can define the angle-normalized error in the sliding window case. The necessity of downdating data of time $t-L$ suggests the use of a hyperbolic type transformation to perform time recursions for SW-ORLS estimation. In [25], the hyperbolic Householder transformation (HHT) was developed to carry out simultaneous addition and deletion of data from least-squares problems. In this section, we show that a single 3×3 hyperbolic Householder transformation carries out both time- and order-update recursions for SW-ORLS estimation.

The angle-normalized errors are defined as, for example

$$\tilde{\mathbf{e}}_x(t) = \Psi(t) \phi_2 \hat{\mathbf{e}}_x(t) \quad (56)$$

and

$$\tilde{\mathbf{e}}_x^z(t) = \tilde{\Psi}(t) \phi_2 \hat{\mathbf{e}}_x^z(t) \quad (57)$$

where $\Psi(t)$ and $\tilde{\Psi}(t)$ are any square matrices that satisfy the relation

$$\Psi^H(t) \phi_2 \Psi(t) = \Phi(t) \quad (58)$$

and

$$\tilde{\Psi}^H(t) \phi_2 \tilde{\Psi}(t) = \tilde{\Phi}(t). \quad (59)$$

Meanwhile, we define scaled correlations as $d(t) = \sqrt{r_{zz}(t)}$ and $\bar{r}(t) = r_{xz}(t)/\sqrt{r_{zz}(t)}$.

Applying a 3×3 hyperbolic Householder transformation $\mathbf{H}$ to a 3×2 matrix obtained by rearranging the scaled correlations and angle normalized errors, we have

$$\mathbf{H} \begin{pmatrix} d(t-1) & \bar{r}(t-1) \\ \tilde{\mathbf{e}}_z(t) & \tilde{\mathbf{e}}_x(t) \end{pmatrix} = \begin{pmatrix} d(t) & -\bar{r}(t) \\ \mathbf{0} & \tilde{\mathbf{e}}_x^z(t) \end{pmatrix}. \quad (60)$$

The hyperbolic Householder transformation matrix $\mathbf{H}$ is given by

$$\mathbf{H} = \phi_3 - [d(t)\delta(t)]^{-1} \mathbf{h} \mathbf{h}^H \quad (61)$$

where $\phi_3 = \text{diag}(1, 1, -1)$, $\delta(t) = d(t) + d(t-1)$, and

$$\mathbf{h} = \begin{pmatrix} \delta(t) \\ \phi_2 \tilde{\mathbf{e}}_z(t) \end{pmatrix}. \quad (62)$$

TABLE I
DIFFERENT FORMS OF LOCAL CELL IMPLEMENTATION

A Posteriori Error Form
Reference Cell $r_{xz}(t) = r_{xz}(t-1) + \mathbf{e}_x^H(t) \Phi^{-1}(t) \mathbf{e}_z(t)$ $\tilde{\Phi}(t) = \Phi(t) - \frac{1}{r_{xz}(t)} \mathbf{e}_x(t) \mathbf{e}_x^H(t)$
Projection Cell $r_{xz}(t) = r_{xz}(t-1) + \mathbf{e}_x^H(t) \Phi^{-1}(t) \mathbf{e}_x(t)$ $\mathbf{e}_x^z(t) = \mathbf{e}_x(t) - k_{xz}(t) \mathbf{e}_x(t)$
A Priori Quotient Form
Reference Cell $r_{xz}(t) = r_{xz}(t-1) + \hat{\mathbf{e}}_x^H(t) \phi_2 \Phi(t) \phi_2 \hat{\mathbf{e}}_x(t)$ $\tilde{\Phi}(t) = \Phi(t) - \frac{1}{r_{xz}(t)} \Phi(t) \phi_2 \hat{\mathbf{e}}_x(t) \hat{\mathbf{e}}_x^H(t) \phi_2 \Phi(t)$
Projection Cell $\hat{\mathbf{e}}_x^z(t) = \hat{\mathbf{e}}_x(t) - k_{xz}(t-1) \hat{\mathbf{e}}_x(t)$ correlation quotient form $r_{xz}(t) = r_{xz}(t-1) + \hat{\mathbf{e}}_x^H(t) \phi_2 \Phi(t) \phi_2 \hat{\mathbf{e}}_x(t)$ direct update (error-feedback) form $k_{xz}(t) = k_{xz}(t-1) - \frac{1}{r_{xz}(t)} \hat{\mathbf{e}}_x(t) \phi_2 \Phi(t) \phi_2 \hat{\mathbf{e}}_x^z(t)$
Angle-Normalized Error Form (HHT Module)
Reference Cell $d(t) = \sqrt{d^2(t-1) + \tilde{\mathbf{e}}_z^H(t) \phi_2 \tilde{\mathbf{e}}_z(t)}$ $\delta(t) = d(t) + d(t-1)$ $\mathbf{h}(t) = \begin{pmatrix} \delta(t) \\ \phi_2 \tilde{\mathbf{e}}_z(t) \end{pmatrix}$
Projection Cell $\mathbf{v}(t) = \begin{pmatrix} \bar{r}(t-1) \\ \tilde{\mathbf{e}}_x(t) \end{pmatrix}$ $\eta(t) = \mathbf{h}^H(t) \mathbf{v}(t) / [d(t)\delta(t)]$ $\bar{r}(t) = -\bar{r}(t-1) + \eta(t)\delta(t)$ $\tilde{\mathbf{e}}_x^z(t) = \phi_2 \tilde{\mathbf{e}}_x(t) - \eta(t) \tilde{\mathbf{e}}_z(t)$

It is clear from (60) that by using a single 3×3 HHT, we project $\tilde{\mathbf{e}}_z(t)$ onto $\tilde{\mathbf{e}}_x(t)$, producing a higher order angle-normalized SWLS estimation error $\tilde{\mathbf{e}}_x^z(t)$; at the same time, we also update in time the scaled correlations $r(t)$, $d(t)$. We can verify (60) by using the basic relations of SW-ORLS estimation established in the previous subsections. We provide the proof of (60) in the Appendix. Since these relations are true for any SW-ORLS estimation, we conclude that time and order updates of any SW-ORLS algorithm can be obtained solely by 3×3 HHT's. This basic building block of QRD-based SW-ORLS algorithms is called the HHT module.

The actual computations involved in HHT module are summarized in Table I, and two other types of local cells are included as well. Each HHT module requires nine multiplications, one division, and one square root. If a sequence of two elementary rotations (one circular and one hyperbolic) are used to perform the hyperbolic transformation, then it requires 14 multiplications, two divisions, and two square roots. Therefore, it is desirable to implement a HHT module when using conventional processors. However, when using

other types of processors, such as a CORDIC processor, it may be advantageous to decompose the 3×3 HHT into a product of a circular rotation and a hyperbolic rotation.

V. HHT-BASED SW-ORLS ALGORITHMS

In Section III, we demonstrated different architectures of SW-ORLS algorithms in the block form. In Section IV, we established the time recursions that lead to the efficient implementations of the basic module. Any combination of an architecture and a variant of local cell implementation is a valid SW-ORLS algorithm.

Applying the HHT module to architectures of ORLS algorithms gives rise to a class of QRD-based SW-ORLS algorithms [24]. The HHT triangular array algorithm is the order-recursive counterpart of the Rader and Steindt's algorithm [25], where HHT is developed to simultaneously update and dowdate a triangular matrix from which the LS solution, i.e., the optimal coefficient vector is obtained. The interpretation of the order-recursive nature of a HHT module makes it possible to extract the final (angle-normalized) residual without computing the optimal coefficient vector explicitly. It also makes it possible to obtain "fast" QRD-based SW-ORLS algorithms, i.e., HHT lattice and HHT multichannel lattice algorithms, when the input data has a shift-invariant or a block shift-invariant property, respectively. In this section, we present the HHT triangular array algorithm and the HHT lattice algorithm in detail.

The triangular array architecture is suitable for array processing applications. To obtain the hyperbolic Householder algorithm in this case, we need to propagate angle-normalized errors $\tilde{\mathbf{e}}_m^{(i)}(t)$ and $\tilde{\mathbf{e}}_m^x(t)$ ($m = 1, \dots, M, i = m+1, \dots, M$). We denote the autocorrelation of residuals $\tilde{\mathbf{e}}_m^{(m+1)}(t)$, which are input to the boundary (reference) cells as $r_m(t) = [\tilde{\mathbf{e}}_m^{(m+1)}(t)]^H \mathbf{W} \tilde{\mathbf{e}}_m^{(m+1)}(t)$ and cross-correlation between these basis residuals with residuals $\tilde{\mathbf{e}}_m^{(i)}(t)$ and $\tilde{\mathbf{e}}_m^x(t)$ propagating in the array as $r_m^{(i)}(t) = [\tilde{\mathbf{e}}_m^{(m+1)}(t)]^H \mathbf{W} \tilde{\mathbf{e}}_m^{(i)}(t)$ and $r_m^x(t) = [\tilde{\mathbf{e}}_m^{(m+1)}(t)]^H \mathbf{W} \tilde{\mathbf{e}}_m^x(t)$, respectively. The time-recursive variables in the hyperbolic Householder triangular array algorithms are defined as $d_m(t) = \sqrt{r_m(t)}$, $\tilde{r}_m^{(i)}(t) = r_m^{(i)}(t)/d_m(t)$, and $\tilde{r}_m^x(t) = r_m^x(t)/d_m(t)$. The hyperbolic Householder triangular array algorithm can be obtained by substituting the quantities in the generic HHT module by the corresponding quantities defined above. We summarize the algorithm in Table II.

We now describe the m th-order lattice stage. The order-recursive variables, i.e., $\tilde{\mathbf{f}}_m(t)$, $\tilde{\mathbf{h}}_m(t)$, and $\tilde{\mathbf{e}}_m(t)$, are the angle-normalized errors. We denote the autocorrelations of forward and backward errors (vector) as $\tilde{r}_m^f(t) = \tilde{\mathbf{f}}_m^H(t) \mathbf{W} \tilde{\mathbf{f}}_m(t)$ and $\tilde{r}_m^b(t) = \tilde{\mathbf{b}}_m^H(t) \mathbf{W} \tilde{\mathbf{b}}_m(t)$. The cross-correlations are defined as $r_m^{fb}(t) = \tilde{\mathbf{b}}_m^H(t-1) \mathbf{W} \tilde{\mathbf{f}}_m(t)$, $r_m^{bf}(t) = \tilde{\mathbf{f}}_m^H(t) \mathbf{W} \tilde{\mathbf{b}}_m(t-1)$, and $r_m^{bx}(t) = \tilde{\mathbf{b}}_m^H(t) \mathbf{W} \tilde{\mathbf{e}}_m^x(t)$. The quantities in the HHT reference cells are defined as $d_m^f(t) = \sqrt{\tilde{r}_m^f(t)}$ and $d_m^b(t) = \sqrt{\tilde{r}_m^b(t)}$. Those in the projection cells are defined as $\tilde{r}_m^{fb}(t) = r_m^{fb}(t)/d_m^b(t-1)$, $\tilde{r}_m^{bf}(t) = r_m^{bf}(t)/d_m^f(t)$, and $\tilde{r}_m^{bx}(t) = r_m^{bx}(t)/d_m^b(t)$. The

TABLE II
HHT TRIANGULAR ARRAY ALGORITHM

Time Initialization:
for $m = 0, \dots, M-1, i = m+2, \dots, M-1$ set:
$\tilde{r}_m^{(i)}(t) = \tilde{r}_m^x(t) = 0, d_m(t) = \epsilon (0 < \epsilon \ll 1)$
Order Initialization:
for $m = 1, \dots, M$ let
$\tilde{\mathbf{e}}_0^{(m)}(t) = \begin{pmatrix} y_m(t) \\ y_m(t-L) \end{pmatrix} \quad \tilde{\mathbf{e}}_0^x(t) = \begin{pmatrix} x(t) \\ x(t-L) \end{pmatrix}$
for $m = 0, \dots, M-1$ do
$d_m(t) = \sqrt{d_m^2(t-1) + [\tilde{\mathbf{e}}_m^{(m+1)}(t)]^H \phi_2 \tilde{\mathbf{e}}_m^{(m+1)}(t)}$
$\delta_m(t) = d_m(t) + d_m(t-1)$
$\mathbf{h}_m(t) = \begin{pmatrix} \delta_m(t) \\ \phi_2 \tilde{\mathbf{e}}_m^{(m+1)}(t) \end{pmatrix}$
for $i = m+2, \dots, M-1$ do
$\mathbf{v}_m^{(i)}(t) = \begin{pmatrix} \tilde{r}_m^{(i)}(t-1) \\ \tilde{\mathbf{e}}_m^{(i)}(t) \end{pmatrix}$
$\eta_m^{(i)}(t) = [\tilde{\mathbf{e}}_m^{(m+1)}(t)]^H \mathbf{v}_m^{(i)}(t) / [d_m(t) \delta_m(t)]$
$\tilde{r}_m^{(i)}(t) = -\tilde{r}_m^{(i)}(t-1) + \eta_m^{(i)}(t) \delta_m(t)$
$\tilde{\mathbf{e}}_{m+1}^{(i)}(t) = \phi_2 [\tilde{\mathbf{e}}_m^{(i)}(t) - \eta_m^{(i)}(t) \tilde{\mathbf{e}}_m^{(m+1)}(t)]$
end do
$\mathbf{v}_m^x(t) = \begin{pmatrix} \tilde{r}_m^x(t-1) \\ \tilde{\mathbf{e}}_m^x(t) \end{pmatrix}$
$\eta_m^x(t) = \mathbf{h}_m^H(t) \mathbf{v}_m^x(t) / [d_m(t) \delta_m(t)]$
$\tilde{r}_m^x(t) = -\tilde{r}_m^x(t-1) + \eta_m^x(t) \delta_m(t)$
$\tilde{\mathbf{e}}_{m+1}^x(t) = \phi_2 [\tilde{\mathbf{e}}_m^x(t) - \eta_m^x(t) \tilde{\mathbf{e}}_m^{(m+1)}(t)]$
end do

hyperbolic Householder lattice stage can be readily obtained by simply substituting the quantities in the generic hyperbolic Householder module by the corresponding quantities in each of the three SWLS angle-normalized order recursions of one lattice stage. We list the hyperbolic Householder lattice algorithm in Table III.

Similarly, we can obtain multichannel hyperbolic Householder lattice algorithms by using the HHT module to implement the local cells. Other cell implementation can be also applied to different architecture to obtain other form of ORLS algorithms.

VI. SUMMARY AND CONCLUSIONS

In this paper, order-recursive least-squares algorithms employing a sliding window are presented. We investigated SW-ORLS algorithms from two independent aspects: global array architecture—how the basic modules are connected to form SW-ORLS estimates of any given order; local cell implementation—how a one-step order update can be implemented by different forms of time-recursive scalar SWLS estimations. Based on the approach outlined in this paper, any form of SW-ORLS algorithms depending on specific applications can easily be obtained. Future work will be directed to further study of the other windowing schemes, especially the so-

TABLE III
HHT LATTICE ALGORITHM

Time Initialization: for $m = 1, \dots, M$ set
$\bar{r}_m^f(-1) = \bar{r}_m^b(-1) = \bar{r}_m^e(-1) = 0$, $d_m^f(-1) = d_m^b(-2) = \epsilon$ ($0 < \epsilon \ll 1$)
Order Initialization:
$\tilde{\mathbf{f}}_0(t) = \tilde{\mathbf{b}}_0(t) = \begin{pmatrix} y(t) \\ y(t-L) \end{pmatrix}$, $\tilde{\mathbf{e}}_0(t) = \begin{pmatrix} x(t) \\ x(t-L) \end{pmatrix}$
for $m = 1, \dots, M$ do
Forward Prediction
$d_m^f(t) = \sqrt{[d_m^f(t-1)]^2 + \tilde{\mathbf{f}}_m^H(t) \phi_2 \tilde{\mathbf{f}}_m(t)}$ $\mathbf{h}_m^f(t) = \begin{pmatrix} \delta_m^f(t) \\ \phi_2 \tilde{\mathbf{f}}_m(t) \end{pmatrix}$
$\delta_m^f(t) = d_m^f(t) + d_m^f(t-1)$
$\mathbf{v}_m^f(t) = \begin{pmatrix} \tilde{\mathbf{r}}_m^f(t-1) \\ \tilde{\mathbf{f}}_m(t) \end{pmatrix}$
$\eta_m^f(t) = [\mathbf{h}_m^f(t)]^H \mathbf{v}_m^f(t) / [d_m^f(t-1) \delta_m^f(t-1)]$
$\bar{r}_m^f(t) = -\bar{r}_m^f(t-1) + \eta_m^f(t) \delta_m^b(t-1)$
$\tilde{\mathbf{f}}_{m+1}(t) = \phi_2 [\tilde{\mathbf{f}}_m(t) - \eta_m^f(t) \tilde{\mathbf{b}}_m(t-1)]$
Backward Prediction
$d_m^b(t) = \sqrt{[d_m^b(t-1)]^2 + \tilde{\mathbf{b}}_m^H(t) \phi_2 \tilde{\mathbf{b}}_m(t)}$ $\mathbf{h}_m^b(t) = \begin{pmatrix} \delta_m^b(t) \\ \phi_2 \tilde{\mathbf{b}}_m(t) \end{pmatrix}$
$\delta_m^b(t) = d_m^b(t) + d_m^b(t-1)$
$\mathbf{v}_m^b(t-1) = \begin{pmatrix} \tilde{\mathbf{r}}_m^b(t-1) \\ \tilde{\mathbf{b}}_m(t-1) \end{pmatrix}$
$\eta_m^b(t) = [\mathbf{h}_m^b(t)]^H \mathbf{v}_m^b(t-1) / [d_m^f(t) \delta_m^f(t)]$
$\bar{r}_m^b(t) = -\bar{r}_m^b(t-1) + \eta_m^b(t) \delta_m^f(t)$
$\tilde{\mathbf{b}}_{m+1}(t) = \phi_2 [\tilde{\mathbf{b}}_m(t-1) - \eta_m^b(t) \tilde{\mathbf{f}}_m(t)]$
Joint Estimation
$\mathbf{v}_m^e(t) = \begin{pmatrix} \tilde{\mathbf{r}}_m^e(t-1) \\ \tilde{\mathbf{e}}_m(t) \end{pmatrix}$
$\eta_m^e(t) = [\mathbf{h}_m^e(t)]^H \mathbf{v}_m^e(t) / [d_m^b(t) \delta_m^b(t)]$
$\bar{r}_m^e(t) = -\bar{r}_m^e(t-1) + \eta_m^e(t) \delta_m^b(t)$
$\tilde{\mathbf{e}}_{m+1}(t) = \phi_2 [\tilde{\mathbf{e}}_m(t) - \eta_m^e(t) \tilde{\mathbf{b}}_m(t)]$
end do

called time-advanced windows [26] and the numerical analysis of the algorithms. It is also desirable to extend SW-ORLS algorithms, which are estimation-error oriented, to applications that need to compute optimal coefficients explicitly, e.g., system identification.

APPENDIX

PROOF OF THE HYPERBOLIC HOUSEHOLDER ORDER-RECURSIVE MODULE GIVEN IN (60)

Let us express the matrix after the hyperbolic Householder transformation as

$$\mathbf{H} \begin{pmatrix} d(t-1) & \bar{r}(t-1) \\ \tilde{\mathbf{e}}_z(t) & \tilde{\mathbf{e}}_x(t) \end{pmatrix} = \begin{pmatrix} r_{11} & r_{12} \\ \mathbf{0} & \mathbf{e}_{22} \end{pmatrix}. \quad (63)$$

By the definition of hyperbolic Householder transformation, it follows that

$$\begin{aligned} |r_{11}|^2 &= [d(t-1) \tilde{\mathbf{e}}_z^H(t)] \phi_3 \begin{pmatrix} d(t-1) \\ \tilde{\mathbf{e}}_z(t) \end{pmatrix} \\ &= r_{zz}(t-1) + \tilde{\mathbf{e}}_z^H(t) \phi_2 \tilde{\mathbf{e}}_z(t) \\ &= r_{zz}(t-1) + \tilde{\mathbf{e}}_z^H(t) \phi_2 \Phi(t) \phi_2 \tilde{\mathbf{e}}_z(t). \end{aligned}$$

According to the time-update recursion of $r_{zz}(t)$ given in (53), it is obvious that

$$r_{11} = \sqrt{r_{zz}(t)} = d(t). \quad (64)$$

To simplify the expression, we shall introduce the following short-hand notation: $\sigma = \sqrt{r_{zz}(t)}$, $\gamma = \sqrt{r_{zz}(t-1)}$, $\beta = \sigma + \gamma$, and

$$\mathbf{h} = \begin{pmatrix} \beta \\ \phi_2 \tilde{\mathbf{e}}_z(t) \end{pmatrix}. \quad (65)$$

Using these variables, we may express the hyperbolic Householder transformation as follows:

$$\mathbf{H} = \phi_3 - \frac{1}{\sigma\beta} \mathbf{h} \mathbf{h}^H. \quad (66)$$

Applying $\mathbf{H}$ expressed by (66) to (63), we may express r_{12} and $\mathbf{e}_{22}$ explicitly.

$$\begin{aligned} r_{12} &= \bar{r}(t-1) \left(1 - \frac{\beta}{\sigma}\right) - \frac{1}{\sigma} \tilde{\mathbf{e}}_z^H(t) \phi_2 \tilde{\mathbf{e}}_x(t) \\ &= -\frac{1}{\sqrt{r_{zz}(t)}} [r_{xz}(t-1) + \tilde{\mathbf{e}}_z^H(t) \phi_2 \Phi(t) \phi_2 \tilde{\mathbf{e}}_x(t)]. \end{aligned}$$

Based on the time update recursion given in (63), it is obvious that

$$r_{12} = -\frac{r_{xz}(t)}{\sqrt{r_{zz}(t)}} = -\bar{r}(t). \quad (67)$$

After the transformation, $d(t)$ and $\bar{r}(t)$ are time updated. Now, let us check the SWLS error after the transformation

$$\begin{aligned} \mathbf{e}_{22} &= \phi_2 \left[\left(\mathbf{I} - \frac{1}{\sigma\beta} \tilde{\mathbf{e}}_z(t) \tilde{\mathbf{e}}_z^H(t) \phi_2 \right) \tilde{\mathbf{e}}_x(t) - \frac{r_{xz}(t-1)}{\sigma\gamma} \right] \\ &= \phi_2 \left[\mathbf{I} - \frac{1}{\sigma\beta} \tilde{\mathbf{e}}_z(t) \tilde{\mathbf{e}}_z^H(t) \phi_2 \right] \\ &\quad \left[\tilde{\mathbf{e}}_x(t) - \frac{1}{\sigma\beta} \left(\mathbf{I} - \frac{1}{\sigma\beta} \tilde{\mathbf{e}}_z(t) \tilde{\mathbf{e}}_z^H(t) \phi_2 \right)^{-1} \tilde{\mathbf{e}}_z(t) \right]. \end{aligned} \quad (68)$$

It is easy to show that

$$\left[\mathbf{I} - \frac{1}{\sigma\beta} \tilde{\mathbf{e}}_z(t) \tilde{\mathbf{e}}_z^H(t) \phi_2 \right]^{-1} = \mathbf{I} + \frac{\frac{1}{\sigma\beta} \tilde{\mathbf{e}}_z(t) \tilde{\mathbf{e}}_z^H(t) \phi_2}{1 - \frac{1}{\sigma\beta} \tilde{\mathbf{e}}_z^H(t) \phi_2 \tilde{\mathbf{e}}_z(t)}. \quad (69)$$

Substituting (69) into (68), we obtain

$$\begin{aligned} \mathbf{e}_{22} &= \phi_2 \left[\mathbf{I} - \frac{1}{\sigma\beta} \tilde{\mathbf{e}}_z(t) \tilde{\mathbf{e}}_z^H(t) \phi_2 \right] \\ &\quad \left[\tilde{\mathbf{e}}_x(t) - \frac{r_{xz}(t-1)}{\sigma\gamma \left(1 - \frac{1}{\sigma\beta} \tilde{\mathbf{e}}_z^H(t) \phi_2 \tilde{\mathbf{e}}_z(t) \right)} \tilde{\mathbf{e}}_z(t) \right]. \end{aligned} \quad (70)$$

By simple manipulation, we identify that

$$r_{zz}(t-1) = \sigma\gamma \left(1 - \frac{1}{\sigma\beta} \tilde{\mathbf{e}}_z^H(t) \phi_2 \tilde{\mathbf{e}}_z(t) \right). \quad (71)$$

Let us also denote

$$\Omega = \phi_2 \left[\mathbf{I} - \frac{1}{\sigma\beta} \tilde{\mathbf{e}}_z(t) \tilde{\mathbf{e}}_z^H(t) \phi_2 \right] \Psi(t). \quad (72)$$

We may rewrite $\mathbf{e}_{22}$ as

$$\mathbf{e}_{22} = \Omega \phi_2 \left[\tilde{\mathbf{e}}_x(t) - \frac{r_{xz}(t-1)}{r_{zz}(t-1)} \tilde{\mathbf{e}}_z(t) \right] = \Omega \phi_2 \tilde{\mathbf{e}}_x^z(t). \quad (73)$$

By simple manipulation, we may obtain

$$\Omega^H \phi_2 \Omega = \Psi^H(t) \phi_2 \Psi(t) - \frac{1}{r_{zz}(t)} (\Psi^H(t) \phi_2 \tilde{\mathbf{e}}_z(t)) (\tilde{\mathbf{e}}_z^H(t) \phi_2 \Psi(t)). \quad (74)$$

By identifying that

$$\begin{aligned} \mathbf{e}_z(t) &= \Phi(t) \phi_2 \hat{\mathbf{e}}_z(t) = \Psi^H(t) \phi_2 \Psi(t) \phi_2 \hat{\mathbf{e}}_z(t) \\ &= \Psi^H(t) \phi_2 \tilde{\mathbf{e}}_z(t) \\ \mathbf{e}_z^H(t) &= \tilde{\mathbf{e}}_z^H(t) \phi_2 \Psi(t) \end{aligned}$$

we may rewrite (74) as

$$\Omega^H \phi_2 \Omega = \Phi(t) - \frac{1}{r_{zz}(t)} \mathbf{e}_z(t) \mathbf{e}_z^H(t) = \tilde{\Phi}(t). \quad (75)$$

Hence, Ω is identical to $\tilde{\Psi}(t)$. Therefore, we may rewrite (73) as

$$\mathbf{e}_{22} = \tilde{\Psi}(t) \phi_2 \hat{\mathbf{e}}_x^z(t) = \tilde{\mathbf{e}}_x^z(t). \quad (76)$$

ACKNOWLEDGMENT

The authors thank the anonymous referees for helpful comments that significantly improve the presentation.

REFERENCES

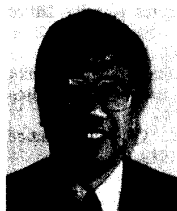
- [1] B. Porat, B. Friedlander, and M. Morf, "Square-root covariance ladder algorithms," *IEEE Trans. Automat. Contr.*, vol. AC-27, pp. 813-829, Aug. 1982.
- [2] G. Carayannis, D. Manolakis, and N. Kalouptsidis, "A fast sequential algorithm for least squares filtering and prediction," *IEEE Trans. Acoust. Speech Signal Processing*, vol. ASSP-31, pp. 1394-1402, Dec. 1983.
- [3] J. M. Cioffi and T. Kailath, "Fast, recursive least squares transversal filters for adaptive filtering," *IEEE Trans. Acoust. Speech Signal Processing*, vol. ASSP-32, pp. 304-337, Apr. 1984.
- [4] D. T. M. Slock and T. Kailath, "Numerically stable fast transversal filters for recursive least-squares adaptive filtering," *IEEE Trans. Acoust. Speech Signal Processing*, vol. 39, pp. 92-114, Jan. 1991.
- [5] G. J. Bierman, *Factorization Methods for Discrete Estimation*. New York: Academic, 1977.
- [6] H. T. Kung and W. M. Gentleman, "Matrix triangularization by systolic arrays," in *Proc. SPIE Real-time Signal Processing IV*, 1981, pp. 19-26, 1981.
- [7] J. G. McWhirter, "Recursive least-squares minimization using a systolic array," *Proc. SPIE*, 1983, paper 431-15.
- [8] F. Ling, "Givens rotation based least-squares lattice and related algorithms," *IEEE Trans. Acoust. Speech Signal Processing*, vol. 39, pp. 1541-1551, July 1991.
- [9] P. A. Regalia and G. Bellanger, "On the duality between fast QR methods and lattice methods in least squares adaptive filtering," *IEEE Trans. Acoust. Speech Signal Processing*, vol. 39, 1991.
- [10] B. Yang and J. F. Böhme, "Rotation-based RLS algorithms: unified derivations, numerical properties, and parallel implementations," *IEEE Trans. Signal Processing*, vol. 40, no. 5, pp. 1151-1167, May 1992.
- [11] S. Ljung and L. Ljung, "Error propagation properties of recursive least-squares adaptation algorithms," *Automatica*, vol. 27, pp. 1-19, 1978.
- [12] K. Zhao, H. Lev-Ari, and J. G. Proakis, "Comparison of order-recursive adaptive RLS algorithms," in *Proc. ICASSP-93* (Minneapolis), Apr. 1993, pp. 416-419, vol. III.
- [13] D. T. Lee, M. Morf, and B. Friedlander, "Recursive least squares ladder estimation algorithms," *IEEE Trans. Acoust. Speech Signal Processing*, vol. ASSP-29, no. 3, pp. 627-641, June 1981.

- [14] H. Lev-Ari, T. Kailath, and J. Cioffi, "Least-squares adaptive lattice and transversal filters: A unified geometric theory," *IEEE Trans. Inform. Theory*, vol. IT-30, no. 2, pp. 222-237, Mar. 1984.
- [15] F. Ling and J. G. Proakis, "A recursive modified Gram-Schmidt algorithm with application to least squares estimation and adaptive filtering," *IEEE Proc. ISCAS'84*, 1984, pp. 781-784.
- [16] I. K. Proudler, J. G. McWhirter, and T. J. Shepherd, "QRD-based lattice filter algorithms," *SPIE Proc.* (San Diego, CA), 1989.
- [17] F. Ling and J. G. Proakis, "A generalized multichannel least squares lattice algorithm based on sequential processing stages," *IEEE Trans. Acoust. Speech Signal Processing*, vol. ASSP-32, pp. 279-287, Apr. 1984.
- [18] H. Lev-Ari, "Modular architectures for adaptive multichannel lattice algorithms," *IEEE Trans. Acoust. Speech Signal Processing*, vol. ASSP-35, no. 4, pp. 543-552, Apr. 1987.
- [19] P. S. Lewis, "QR-based algorithms for multichannel adaptive least squares lattice filters," *IEEE Trans. Acoust. Speech Signal Processing*, vol. 38, no. 3, pp. 421-432, Mar. 1990.
- [20] K. K. Zhao, F. Ling, and J. G. Proakis, "Multichannel Givens lattice adaptive algorithm," in *Proc. ICASSP-91*, May 1991, pp. 1849-1852.
- [21] F. Ling, J. G. Proakis, and K. K. Zhao, "A systematic treatment of order-recursive least-squares algorithms," *Proc. SPIE Adaptive Signal Processing*, 1991.
- [22] S. Haykin, *Adaptive Filter Theory*, Englewood Cliffs, NJ: Prentice-Hall, 1991, 2nd ed.
- [23] F. Ling and J. G. Proakis, "Numerical accuracy and stability: Two problems of adaptive estimation algorithms caused by round-off error," in *Proc. ICASSP 83* (San Diego, CA), 1983.
- [24] K. K. Zhao, F. Ling, H. Lev-Ari, and J. G. Proakis, "QRD-based sliding window adaptive LS lattice algorithms," in *Proc. ICASSP-92* (San Francisco), Mar. 1992, pp. IV-49.
- [25] C. M. Rader and A. O. Steinhardt, "Hyperbolic Householder transformations," *IEEE Trans. Acoust. Speech Signal Processing*, vol. ASSP-34, no. 6, pp. 1589-1602, Dec. 1986.
- [26] P. Strobach, "New forms of Levinson and Schur algorithms," *IEEE Signal Processing Mag.*, vol. 8, no. 1, pp. 12-36, Jan. 1991.



Karl K. Zhao (S'89-M'93) received the B.S. degree from Tsinghua University, Beijing, China, in 1988 and the M.S. and Ph.D. degrees from Northeastern University, Boston, MA, in 1990 and 1992, respectively, all in electrical engineering.

He was with Northeastern University from September 1988 to June 1992 as a Teaching Assistant and as a Research Assistant. Since May 1992, he has been with DSP Software Engineering Inc., Bedford, MA, as a senior design engineer, where he is responsible for the research, design, and development of advanced digital signal processing algorithms for various telecommunication products such as high-speed software modems for data and facsimile transmission, acoustic echo cancellers, and low bit-rate vocoders. His research interests include adaptive filtering and estimation theory and its applications in digital communications for both wired and wireless links, acoustic echo cancellation, active noise control, adaptive beamforming, multirate signal processing, and highly concurrent and parallel architectures for high throughput rate digital signal processing applications.



Fuyun Ling (SM'87) received the B.S. degree from the Radio-Electronics Department, Tsinghua University, Beijing, China, in 1967 and the M.S. and Ph.D. degrees from the Electrical and Computer Engineering Department, Northeastern University, Boston MA, in 1981 and 1984, respectively, all in electrical engineering.

From 1968 to 1978, he was an Electrical Engineer at the Chaoyan Heavy-Machinery Factory, Liaoning, China. From 1979 to 1984, he was with Northeastern University, where he served as a Teaching Assistant and an NSF Research Fellow. From 1984 to October 1992, he was employed by Motorola Codex, Canton, MA, in the Department of Research and Advanced Development, where he was responsible for the research, design, and development of architecture, software, and algorithms of various Codex data transmission and networking products, in particular, high-speed echo cancellation modems. In November 1992, he was transferred to Motorola Corporate R&D Center, Schaumburg, IL, where he is a Principal Staff Engineer in the Communication System Research Laboratory and is working on analysis, implementation, and system design of mobile and personal communication systems. Since 1987, he has been an Adjunct Professor at Northeastern University. He holds nine U.S. Patents with a number of other patent applications pending. He has published over 50 technical papers and is a coauthor of a textbook and a contributor to another book, all in the field of digital communication and digital signal processing. His research interests have centered on digital communication and digital signal processing, including personal and mobile communication systems, wired and wireless data transmission, theory and algorithms of adaptive estimation, multirate digital signal processing, and applications of digital signal processing to communications.

Dr. Ling serves as an Associate Editor for the IEEE TRANSACTIONS ON SIGNAL PROCESSING (1992–1994). He is a member of the Motorola Scientific Advisory Board Associates (SABA) and a member of Sigma Ki, Tau Beta Pi, and Phi Kappa Phi.



Hanoeh Lev-Ari (S'78–M'84–SM'93) received the B.Sc. degree, Summa Cum Laude, in 1971 and the M.Sc. degree in 1978, both in electrical engineering, from the Technion, Israel Institute of Technology, Haifa, Israel; he received the Ph.D. degree in electrical engineering from Stanford University, Stanford, CA, in 1984.

He served in the Israel Defense Forces from 1971 to 1976 as an electronic systems engineer. From 1982 to 1984, he was a Research Scientist with Integrated Systems, Inc., Palo Alto, CA. During 1985, he held a joint appointment as an Adjunct Research Professor of Electrical Engineering with the Naval Postgraduate School, Monterey, CA, and as a Research Associate with the Information Systems Laboratory at Stanford; he stayed at Stanford as a Senior Research Associate until 1990. He is currently an Associate Professor with the Department of Electrical and Computer Engineering at Northeastern University, Boston, MA. His present areas of interest include model-based spectrum analysis and estimation for nonstationary signals, wavelet representation of random signals, and adaptive linear and nonlinear prediction techniques, with applications to channel equalization, over-the-horizon (OTH) radar, and array processing of high-frequency signals. His past research has involved a number of mathematical techniques and a variety of applications in signal processing and linear systems, including lossless cascade models for multiple-input/multiple-output systems; extension of maximum-entropy techniques to multidimensional signal processing; modeling, analysis, and design of parallel algorithms and architectures; and characterization of structured matrices.

Dr. Lev-Ari served as an Associate Editor of *Circuits, Systems, and Signal Processing* and of *Integration, The VLSI Journal*. He is a member SIAM.



John G. Proakis (F'84) received the E.E. degree from the University of Cincinnati in 1959, the S.M. degree in electrical engineering from the Massachusetts Institute of Technology (MIT), Cambridge, in 1961, and the Ph.D. degree from Harvard University, Cambridge, MA, in 1966.

From June 1959 to September 1963, he was associated with MIT, first as a research assistant and later as a staff member at the Lincoln Laboratory, Lexington, MA. During the period 1963–1966, he was engaged in graduate studies at Harvard University, where he was a research assistant in the Division of Engineering and Applied Physics. In December 1966, he joined the staff of the Communication Systems Laboratories of Sylvania Electronic Systems and later transferred to the Waltham Research Center of General Telephone and Electronics Laboratories, Inc., Waltham, MA. Since September 1969, he has been with Northeastern University, Boston, MA, where he holds the rank of Professor of Electrical Engineering. From July of 1982 to June of 1984, he held the position of Associate Dean of the College of Engineering and Director of the Graduate School of Engineering. Since July of 1984, he has been Chairman of the Department of Electrical and Computer Engineering. His interests have centered on digital communications, spread spectrum systems, system modeling and simulation, adaptive filtering, and digital signal processing. He is the author of the book *Digital Communications* (New York: McGraw-Hill, 1983, 1st ed.; 1989, 2nd ed.) and co-author of the books *Digital Signal Processing* (New York: MacMillan, 1992, 2nd ed.), *Digital Signal Processing Laboratory* (Englewood Cliffs, NJ: Prentice-Hall, 1991), *Advanced Digital Signal Processing* (New York: MacMillan, 1992), and *Digital Processing of Speech Signals* (New York: MacMillan, 1993).

Dr. Proakis has served as an Associate Editor for the IEEE TRANSACTIONS ON INFORMATION THEORY (1974–1977) and the IEEE TRANSACTIONS ON COMMUNICATIONS (1973–1974). He has also served on the Board of Governors of the Information Theory Group (1977–1983), is a Registered Professional Engineer in the State of Ohio, and is a member of Eta Kappa Nu, Tau Beta Pi, and Sigma Xi.